



Presentación para la Defensa del TFG

Comanda Digital — Sistema de Gestión de Comandas

Guión de Diapositivas · 15 minutos

Cómo usar este documento: Cada sección es una diapositiva. El texto en *cursiva* es lo que dices en voz alta. Las viñetas son el contenido visual.

- Título grande: **Comanda Digital**
- Subtítulo: *Sistema de Gestión de Comandas para Restaurantes*
- Tu nombre · Ciclo Formativo Grado Superior · Mayo 2026
- Logo/icono de restaurante

✦ Contenido visual

🗣️ Lo que dices:

"Buenos días. Mi proyecto se llama Comanda Digital, y es una aplicación web para digitalizar el proceso completo de gestión de pedidos en restaurantes, desde que el cliente se sienta hasta que se genera el ticket de cobro."

- Título: **El problema**
- Imagen o icono de papel y bolígrafo (comanda tradicional)
- Tres puntos:
 - ✖ Comandas en papel → errores de transcripción
 - ✖ Comunicación verbal con cocina → demoras y malentendidos
 - ✖ Cartas QR en PDF → estáticas, sin alérgenos ni carrito

✦ Contenido visual

🗣️ Lo que dices:

"La mayoría de restaurantes siguen gestionando los pedidos en papel. Cuando existe tecnología, suele ser una carta en PDF accesible por QR, que no resuelve el problema real: la comunicación entre sala, cocina y barra. Eso es lo que Comanda Digital soluciona."

- Título: **La solución**
- Diagrama simple con 4 actores conectados:
 - 🧑 Cliente → (QR) → 💻 Sistema
 - 💻 Sistema → 👨🍳 Cocina / 🍹 Barra
 - 💻 Sistema → 🎫 Ticket generado automáticamente
- Tagline:

🌟 *Contenido visual*

🗣️ **Lo que dices:**

"Un sistema completo: cliente, cocina, barra y administración en una sola plataforma"

"Comanda Digital conecta al cliente, que hace el pedido desde su móvil escaneando el QR de la mesa, con los paneles de cocina y barra que reciben los pedidos en tiempo real. Cuando toca pagar, el sistema genera el ticket automáticamente con el desglose del IVA."

- Título: **Stack tecnológico**
- Tabla o grid con iconos:

Capa	Tecnología	Por qué
Backend	Symfony 8	Framework PHP robusto, seguridad integrada
Frontend	React 18 + Twig	React para interfaces dinámicas, Twig para layout y auth
Estilos	Tailwind CSS	Mobile First, sistema de diseño consistente
Base de datos	MariaDB 11.3	RDBMS relacional, ACID
Infraestructura	Docker + Nginx	Entorno reproducible, despliegue en 1 comando

🌟 *Contenido visual*

🗣️ **Lo que dices:**

"El backend está construido con Symfony 8, que gestiona la seguridad, el enrutado y la lógica de negocio. Para el frontend usé una arquitectura híbrida: Twig gestiona el layout y la autenticación, y React 18 gestiona las interfaces dinámicas como el menú del cliente o los paneles de cocina. Todo bajo Tailwind CSS con diseño Mobile First."

- Título: **Arquitectura del sistema**
- Diagrama de capas (de arriba hacia abajo):



✦ Contenido visual

🗣️ Lo que dices:

"La arquitectura tiene tres capas: Nginx como proxy inverso en el puerto 8001, Symfony con PHP-FPM gestionando la lógica, y MariaDB como base de datos. Los tres servicios corren en contenedores Docker y se levantan con un único comando: `docker compose up`."

- Título: **Decisión de diseño: React embebido en Twig**
- Dos columnas:
 - ☒ **Ventajas de esta arquitectura:**
 - Autenticación y seguridad 100% en Symfony
 - Enrutado centralizado
 - React solo donde aporta valor (interfaces dinámicas)
 -  **Cómo funciona:**
 - Twig renderiza la página con `{{ react_component('MenuPage', { datos }) }}`
 - React recibe los datos iniciales y gestiona el estado local

 *Contenido visual*

Lo que dices:

"Una decisión importante fue no hacer una SPA pura. Mantener Symfony como guardián de la autenticación simplifica enormemente la seguridad. React entra solo en las vistas que necesitan reactividad: el menú del cliente, los paneles de cocina y barra, y el panel de administración."

- Título: **Modelo de datos — Entidades principales**
- Diagrama simplificado (solo las relaciones clave):



- Nota: **11 entidades, 11 migraciones**

 *Contenido visual*

Lo que dices:

"El modelo de datos tiene 11 entidades. El núcleo es Mesa-Pedido-DetallePedido. Cuando el cliente hace un pedido, el sistema separa automáticamente los items por tipo de categoría: los platos van al panel de cocina y las bebidas al panel de barra, cada uno con su propio estado."

- Título: **Separación automática por destino**
- Diagrama del flujo de separación:

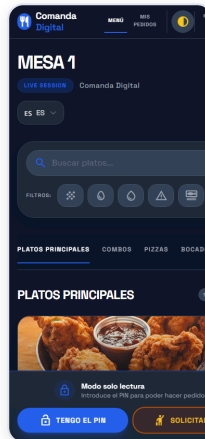


✦ Contenido visual

🔧 Lo que dices:

"Esta separación automática es una de las piezas clave del sistema. El controller detecta el tipo de cada categoría y crea un pedido independiente para cocina y otro para barra. Así cada panel gestiona sus estados de forma totalmente independiente."

- Título: **Demo — Menú del cliente**
- Screenshot o captura del menú en móvil
- Puntos destacados señalados con flechas:
 - Filtro de alérgenos
 - Carrito flotante
 - Categorías en navegación



Menú del cliente — vista móvil

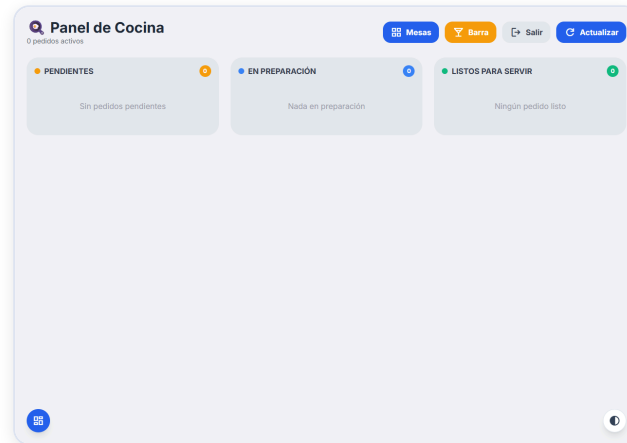
✦ Contenido visual

✦ Abrir la aplicación en el navegador, ir a la URL de una mesa

🔑 Lo que dices:

"Esto es lo que ve el cliente al escanear el QR. La carta se carga directamente en el navegador del móvil, sin instalar ninguna app. Puede filtrar por alérgenos — si marca 'Gluten', los productos no aptos desaparecen. Añade al carrito con un toque y cuando está listo confirma el pedido."

- Título: **Demo — Panel de cocina / barra**
- Screenshot del panel Kanban
- Leyenda del semáforo:
 - Verde: menos de 5 min
 - Amarillo: 5-10 min
 - Rojo: más de 10 min



Panel de cocina — sistema Kanban

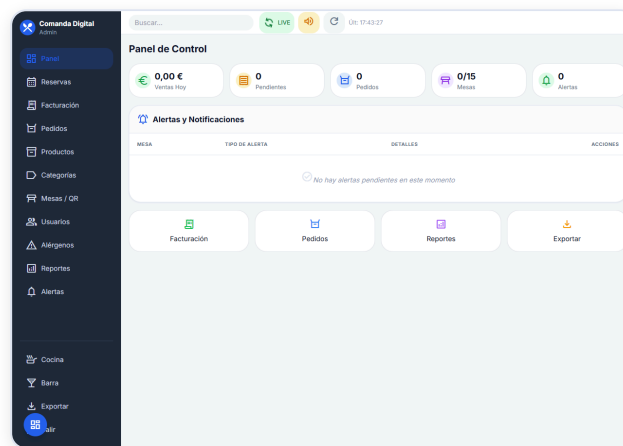
✦ *Contenido visual*

✦ *Ir al panel de cocina — mostrar el pedido que acaba de llegar*

🔑 **Lo que dices:**

"En cuanto el cliente confirma, el pedido aparece aquí en verde. El sistema hace polling cada 10 segundos. El cocinero toca la tarjeta para pasar de pendiente a en preparación, y cuando termina marca listo. Cuando supera los 10 minutos sin atender, pasa a rojo — el semáforo."

- Título: **Demo — Administración**
- Lista de funcionalidades del panel admin:
 - CRUD de productos, categorías, alérgenos
 - Gestión de mesas (QR generado automáticamente)
 - Gestión de usuarios con roles
 - Sistema de reservas
 - Reportes y tickets



Panel de administración

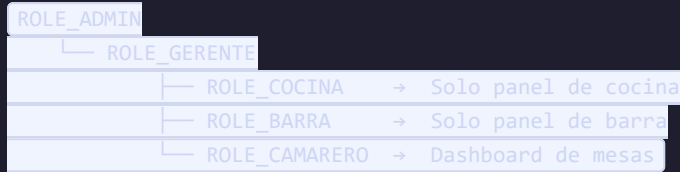
✦ *Contenido visual*

✦ *Ir al panel de administración — mostrar CRUD de productos y gestión de mesas*

Lo que dices:

"El panel de administración lo tienen el admin y el gerente. Desde aquí gestionan el catálogo completo, las mesas, los usuarios con sus roles, las reservas y los reportes. Cuando se cierra una mesa, el sistema genera automáticamente el ticket con el desglose del IVA al 10%."

- Título: **Seguridad — Control de acceso por roles**
- Jerarquía visual:



- Puntos adicionales:
 - Contraseñas hasheadas con Argon2
 - PIN de mesa regenerado al cerrar
 - CSRF token en el login

📌 Contenido visual

🔑 Lo que dices:

"La seguridad está gestionada completamente por Symfony. Cada rol solo puede acceder a su área. Un barman no puede entrar al panel de admin, y un cocinero no ve los pedidos de barra. Las contraseñas están hasheadas y el PIN de cada mesa se regenera automáticamente al cerrarla, para que el siguiente cliente no pueda usar la sesión anterior."

- Título: **Despliegue reproducible con Docker**
- Bloque de código en tipografía monoespaciada:

```
docker compose up -d
```

- Los tres servicios que levanta:
 - `database` — MariaDB 11.3 (con healthcheck)
 - `app` — PHP-FPM + Symfony (instala deps, aplica esquema, carga datos)
 - `web` — Nginx (proxy inverso)
- Resultado: aplicación lista en `http://localhost:8001`

📌 *Contenido visual*

🔑 **Lo que dices:**

"El despliegue completo se hace con un único comando. Docker Compose levanta los tres contenedores en orden correcto: primero la base de datos, que tiene healthcheck para confirmar que está lista, luego la aplicación que instala dependencias, aplica el esquema y carga los datos de demo, y por último Nginx. Todo automático."

- Título: **Qué he aprendido**
- Tres columnas o secciones:
 - **Técnico:**
 - Arquitectura MVC real en producción
 - Integración Symfony + React
 - Seguridad por capas (roles, PIN, CSRF, hashing)
 - Contenerización con Docker
 - **De proceso:**
 - Importancia de separar responsabilidades (controllers ligeros)
 - El valor de las migraciones para evolucionar el esquema
 - Gestión de estados complejos con React
 - **De producto:**
 - La UX Mobile First es crítica cuando el usuario es el cliente final
 - El polling funciona; WebSockets es mejor, pero el MVP es válido

✦ *Contenido visual*

🗣️ **Lo que dices:**

"Este proyecto me ha permitido trabajar con un stack tecnológico real de nivel profesional. Lo más valioso ha sido entender cómo encajan las piezas: Symfony como capa de seguridad y lógica, React para la UX dinámica y Docker para garantizar que el entorno es siempre el mismo."

- Título: **Líneas de mejora**
- Lista con iconos:
 - 🚀 **WebSockets / Mercure** — Notificaciones push en tiempo real (eliminar polling)
 - 💳 **Integración TPV real** — Conexión con terminales de punto de venta
 - 📱 **App nativa** — React Native para el personal de sala
 - 📊 **Analytics avanzados** — Dashboard con métricas de negocio en tiempo real
 - 🌐 **Multi-restaurante** — Soporte para cadenas con un mismo panel
 - ✅ **Tests automatizados** — Cobertura completa con PHPUnit

🌟 *Contenido visual*

🗣️ **Lo que dices:**

"Las principales mejoras pendientes son WebSockets para eliminar el polling, integración con TPV real y una aplicación nativa para el personal. El sistema actual es un MVP funcional y desplegable; estas mejoras lo convertirían en un producto comercialmente viable."

- Título grande: **Comanda Digital**
- Subtítulo: *¿Preguntas?*
- QR o URL de acceso a la demo
- Tu nombre y datos de contacto

🌟 *Contenido visual*

🗣️ **Lo que dices:**

"Eso es todo. Comanda Digital es una aplicación completa, desplegable con Docker, con seguridad por roles, separación de pedidos por destino y panel de administración. Quedo a vuestra disposición para las preguntas."

Preguntas Frecuentes del Tribunal

Preparación para posibles preguntas durante la defensa.

Arquitectura y código

¿Por qué usaste `doctrine:schema:update` en vez de las migraciones en el endpoint Docker?

Por simplicidad en el despliegue inicial. Las migraciones existen y se usan durante el desarrollo, pero para el despliegue de demostración `schema:update --force` garantiza que el esquema siempre está sincronizado con las entidades independientemente del estado de las migraciones. En producción real usaría `doctrine:migrations:migrate`.

¿Cómo garantizas que no hay SQL Injection?

Doctrine ORM usa prepared statements en todas las consultas. Nunca concateno SQL manualmente; uso el QueryBuilder o los repositorios de Doctrine, que parametrizan automáticamente los valores.

¿Por qué polling y no WebSockets?

El polling cada 10 segundos es un compromiso razonado. Para el contexto de un restaurante, una latencia de 10 segundos es aceptable y la implementación es mucho más simple: no requiere servidor de eventos ni configuración extra de Docker. WebSockets es la mejora natural, y está identificada como trabajo futuro.

¿Cómo funciona el PIN de la mesa?

Cada mesa tiene un PIN de 8 dígitos generado con `random_int()`. Cuando el cliente hace un pedido, debe incluir ese PIN. Cuando la mesa se cierra, el PIN se regenera automáticamente con `regeneratePin()`, de modo que el siguiente cliente no puede usar la sesión anterior.

Base de datos

¿Por qué guardas el precio en `DetallePedido` y no solo el ID del producto?

Es un snapshot del precio en el momento del pedido. Si el administrador cambia el precio de un producto mañana, los tickets históricos deben seguir mostrando el precio que pagó el cliente. Es un patrón estándar en sistemas de facturación.

¿Qué pasa si dos mesas hacen un pedido al mismo tiempo?

Doctrine gestiona la concurrencia a nivel de transacción. Cada pedido se guarda en su propia transacción. El único punto donde podría haber concurrencia es en la numeración del ticket, que está resuelto: el número se asigna tras el primer `flush()` usando el `id` del ticket generado por la base de datos, que es único e incremental.

Docker y despliegue

¿Qué pasa si el tribunal ejecuta `docker compose up` en una máquina sin internet?

Las imágenes de Docker se descargan la primera vez. Si la máquina ya las tiene cacheadas (como ocurrirá en la defensa tras una primera ejecución), el levantamiento es completamente offline.

¿Cómo persisten los datos entre reinicios del contenedor?

Los datos de MariaDB se guardan en un volumen Docker llamado `database_data`. Aunque el contenedor se destruya y se cree, el volumen persiste. Solo se borran los datos si se ejecuta `docker compose down -v`.

Seguridad

¿Cómo está protegida la ruta `/admin` de un usuario sin permisos?

En `security.yaml` hay una regla `access_control` que exige `ROLE_ADMIN` para cualquier ruta que empiece por `/admin`. Symfony intercepta la petición antes de que llegue al controller y redirige al login si el usuario no está autenticado o no tiene el rol requerido.

¿Qué pasaría si alguien intenta hacer un pedido sin QR ni PIN?

El controller verifica el PIN antes de procesar cualquier pedido. Si el PIN no coincide con el almacenado en la mesa, devuelve un 401 Unauthorized. Sin el token de la mesa en la URL tampoco puede acceder a la carta, porque el controller busca la mesa por ese token.